



Servicio Nacional de Aprendizaje – SENA

Tecnólogo en Análisis y Desarrollo de Software
Ficha: 3134555

**Elabora documentos técnicos y de usuario del software
(GA10-220501097-AA10-EV01)**

APRENDIZ
Johan Mauricio Sánchez Gaviria

INSTRUCTOR
Alfonso González Vargas

JUNIO 2026

Contenido

1. Introducción	4
2. Objetivo.....	4
3. Alcance del manual técnico	4
4. Descripción general del sistema.....	4
5. Prerrequisitos de instalación del sistema	5
6. Frameworks, herramientas y estándares	6
7. Estructura del repositorio.....	6
8. Arquitectura técnica	6
9. Diagrama de casos de uso del sistema	8
10. Modelo entidad relación de la base de datos	8
11. Diccionario de datos	9
Tabla usuarios	9
Tabla clientes	10
Tabla productos.....	10
Tabla ventas	10
Tabla detalle_venta	10
12. Scripts de instalación y configuración	11
13. Configuración del backend	12
14. Servicios API REST	12

15. Seguridad y control de roles	13
16. Componentes backend	13
17. Componentes frontend	14
18. Procedimiento técnico de instalación local	14
19. Pruebas técnicas recomendadas	15
20. Consideraciones de soporte y mantenimiento	15
21. Relación con la documentación de usuario	15
22. Conclusiones	16
23. Bibliografía	16

1. Introducción

El presente manual técnico documenta la estructura, arquitectura, componentes, configuración, instalación y operación técnica del sistema Alumipro, aplicación desarrollada para apoyar la gestión interna de una empresa colombiana del sector aluminio. El sistema permite administrar usuarios, clientes, productos, ventas, histórico de ventas y control de inventario, integrando una aplicación web con backend API REST, base de datos relacional y aplicación móvil Android alineada con el proyecto formativo.

2. Objetivo

Documentar técnicamente el sistema Alumipro para facilitar su instalación, mantenimiento, comprensión arquitectónica, despliegue local, administración de base de datos y soporte durante la fase de implantación.

3. Alcance del manual técnico

Este manual cubre la aplicación web de Alumipro, su backend en Spring Boot, frontend en React, base de datos MySQL/MariaDB, API REST, seguridad con JWT, estructura de carpetas, ejecución local, scripts SQL, pruebas técnicas y relación con el cliente móvil Android. No reemplaza el manual de usuario final; este documento está dirigido a desarrolladores, administradores técnicos, personal de soporte y responsables de implantación.

4. Descripción general del sistema

Alumipro es una solución de gestión interna para una empresa especializada en fabricación y comercialización de perfiles de aluminio, ventanería arquitectónica, fachadas y estructuras. El

sistema centraliza las operaciones de inventario, clientes, ventas y usuarios, aplicando autenticación, control de roles y validaciones de negocio.

Módulo	Descripción técnica
Autenticación	Ingreso mediante correo y contraseña. El backend genera token JWT para proteger los servicios internos.
Roles	ADMIN y VENDEDOR. El rol ADMIN administra usuarios y productos; el vendedor registra ventas y consulta información operativa.
Clientes	Consulta, creación, edición y eliminación controlada de clientes.
Productos	Gestión de inventario, precio, descripción y disponibilidad de stock.
Ventas	Registro de ventas con cliente, productos, cantidades, totales y descuento automático de stock.
Histórico	Consulta consolidada de ventas y detalle por transacción.
App móvil	Aplicación Android que consume servicios REST y presenta información operativa relacionada con Alumipro.

5. Prerrequisitos de instalación del sistema

Componente	Requisito
Sistema operativo	Windows 11 o sistema compatible para desarrollo Java/Node/MySQL.
Java	JDK 17 instalado y configurado en PATH.
Maven	Maven compatible con Spring Boot para compilar, probar y ejecutar el backend.
Node.js y npm	Node.js 24 o versión compatible con el frontend React/Vite.
Base de datos	XAMPP con MySQL/MariaDB o MySQL 8 compatible. Puerto por defecto: 3306.
IDE recomendado	IntelliJ IDEA para backend, Visual Studio Code para frontend, Android Studio para app móvil.
Git	Control de versiones para clonar, versionar y publicar cambios.
Navegador	Chrome, Edge o Firefox para consumir la interfaz web.

6. Frameworks, herramientas y estándares

Área	Frameworks / estándares
Backend	Java 17, Spring Boot 2.7.18, Spring Web MVC, Spring Data JPA, Spring Security, Maven.
Seguridad	JWT con biblioteca JJWT 0.11.5, BCryptPasswordEncoder para contraseñas, filtro JWT por petición, autorización por roles.
Persistencia	JPA/Hibernate, MariaDB JDBC Driver 3.1.4, dialecto MariaDB103Dialect, motor InnoDB, llaves foráneas e índices.
Frontend	React 19, TypeScript 5.9, Vite 7.3, React Router, CSS Modules, lucide-react, react-hot-toast.
Pruebas	JUnit 5, Mockito, Spring Boot Test, Node Test Runner, TypeScript, ESLint.
Base de datos	MySQL/MariaDB, codificación utf8mb4, scripts schema.sql, seed.sql y patch_auth_ventas.sql.
Móvil	Android Studio, Java, XML, SQLite y consumo de API REST.
Estándares aplicados	Arquitectura por capas, API REST JSON, separación frontend/backend, control de roles, validación de entradas y documentación técnica.

7. Estructura del repositorio

Ruta	Contenido
apps/backend	Proyecto Maven con API REST, entidades, repositorios, servicios, seguridad y pruebas backend.
apps/frontend	Proyecto React/Vite con páginas, componentes, servicios HTTP, rutas protegidas, estilos y pruebas frontend.
db	Scripts SQL de creación de base de datos, datos iniciales y parches técnicos.
docs	Documentación técnica, colección Postman y guías de ejecución.
Alumipro_INC_mobile	Aplicación móvil Android alineada funcionalmente con Alumipro.

8. Arquitectura técnica

Alumipro está organizado bajo una arquitectura por capas. La capa de presentación corresponde al frontend React y la app Android; la capa de servicios expone una API REST en Spring Boot;

la capa de negocio se implementa mediante servicios Java; la capa de persistencia usa repositorios JPA; y la información se almacena en MySQL/MariaDB.

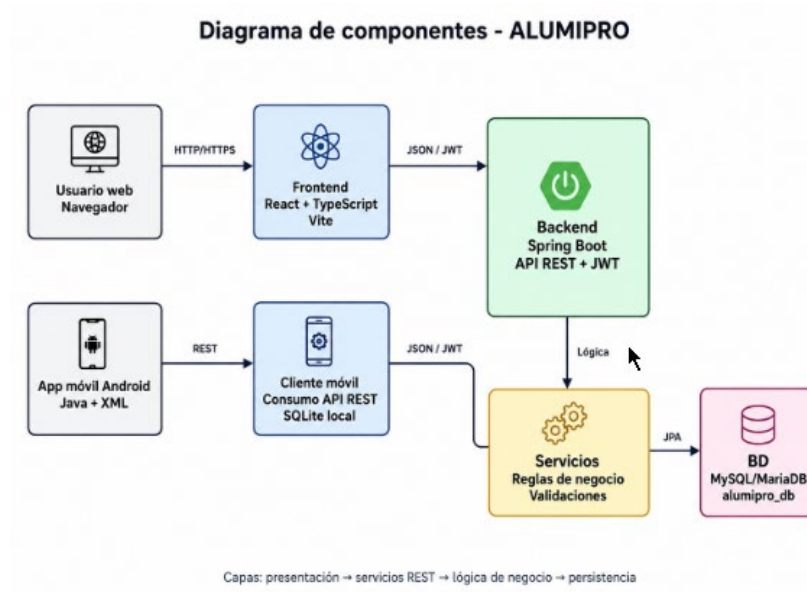
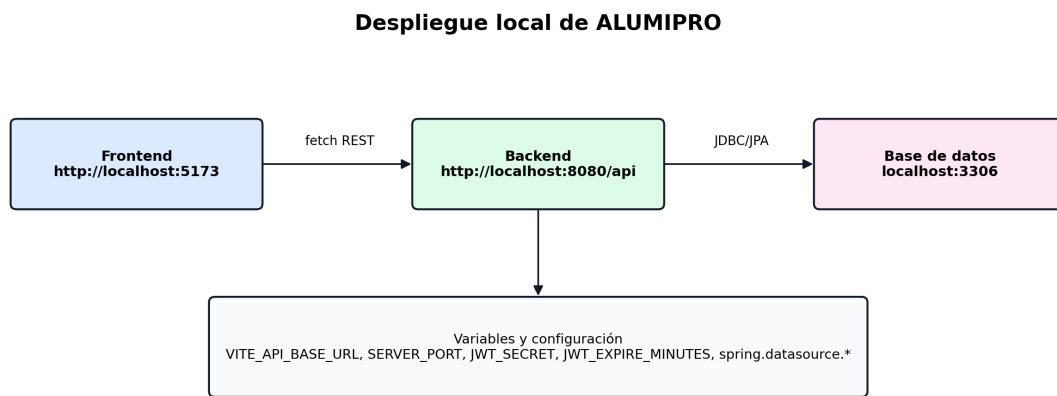


Figura 1. Diagrama de componentes del sistema Alumipro.



Ambiente usado en pruebas: Windows 11, XAMPP/MySQL, IntelliJ IDEA, Node.js y Git.

Figura 2. Despliegue local de ALUMIPRO.

9. Diagrama de casos de uso del sistema

Los actores principales son Administrador y Vendedor. Ambos pueden ingresar al sistema, consultar módulos operativos y registrar ventas; el Administrador posee permisos adicionales para gestionar productos, usuarios y acciones administrativas restringidas.

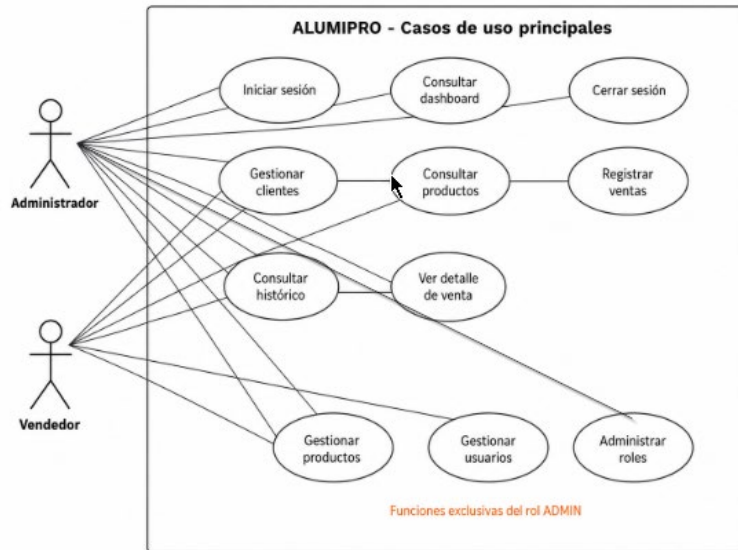


Figura 3. Diagrama de casos de uso principales de Alumipro.

Actor	Responsabilidad
Administrador	Acceso completo: dashboard, clientes, productos, ventas, histórico, detalle de ventas y usuarios.
Vendedor	Acceso operativo: dashboard, clientes, consulta de productos, registro de ventas e histórico. No administra usuarios ni crea productos.

10. Modelo entidad relación de la base de datos

La base de datos principal se denomina Alumipro_db. El modelo relacional se compone de usuarios, clientes, productos, ventas y detalle_venta. Las ventas se relacionan con clientes y

usuarios vendedores; cada venta posee uno o varios detalles, y cada detalle referencia un producto.

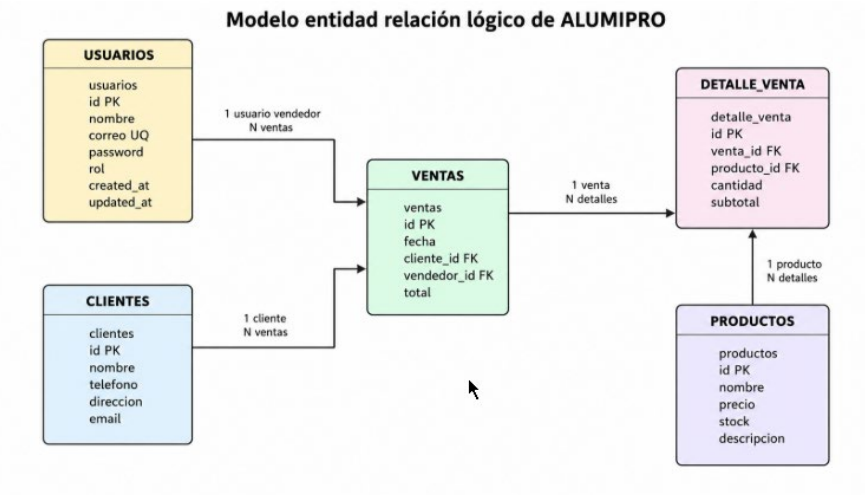


Figura 4. Modelo entidad relación lógico de Alumipro.

11. Diccionario de datos

El siguiente diccionario resume los campos principales definidos en el script db/schema.sql.

Tabla usuarios

Campo	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador del usuario.
nombre	VARCHAR(120)	NOT NULL	Nombre visible del usuario.
correo	VARCHAR(120)	NOT NULL, UNIQUE	Correo usado para autenticación.
password	VARCHAR(255)	NOT NULL	Contraseña cifrada con BCrypt.
rol	ENUM(ADMIN, VENDEDOR)	NOT NULL	Rol funcional y de seguridad.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de creación.
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Fecha de actualización.

Tabla clientes

Campo	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador del cliente.
nombre	VARCHAR(100)	NOT NULL	Nombre del cliente.
telefono	VARCHAR(20)	NULL	Teléfono de contacto.
direccion	VARCHAR(160)	NULL	Dirección del cliente.
email	VARCHAR(100)	NULL	Correo de contacto.

Tabla productos

Campo	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador del producto.
nombre	VARCHAR(255)	NOT NULL	Nombre del producto.
precio	DECIMAL(12,2)	NOT NULL	Valor unitario.
stock	INT	NOT NULL	Cantidad disponible.
descripcion	TEXT	NULL	Descripción técnica o comercial.

Tabla ventas

Campo	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador de venta.
fecha	DATE	NOT NULL	Fecha de registro.
cliente_id	INT	FK clientes(id)	Cliente asociado.
vendedor_id	INT	FK usuarios(id), NULL	Usuario vendedor asociado.
total	DECIMAL(12,2)	NOT NULL	Total de la venta.

Tabla detalle_venta

Campo	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador del detalle.
venta_id	INT	FK ventas(id)	Venta asociada.
producto_id	INT	FK productos(id)	Producto vendido.
cantidad	INT	NOT NULL	Cantidad vendida.

subtotal	DECIMAL(12,2)	NOT NULL	Subtotal por ítem.
----------	---------------	----------	--------------------

12. Scripts de instalación y configuración

Los scripts de instalación se encuentran en la carpeta db del repositorio. El orden recomendado de ejecución es schema.sql, seed.sql y, cuando aplique actualización de estructura, patch_auth_ventas.sql.

Archivo	Propósito
db/schema.sql	Crea la base Alumipro_db, tablas, llaves primarias, llaves foráneas e índices.
db/seed.sql	Inserta datos iniciales de clientes y productos para pruebas funcionales.
db/patch_auth_ventas.sql	Aplica ajustes de usuarios, roles, vendedor_id y restricciones relacionadas con autenticación y ventas.
docker-compose.yml	Permite levantar un contenedor MySQL 8 con scripts de inicialización de base de datos.

Fragmento base de db/schema.sql

```
CREATE DATABASE IF NOT EXISTS Alumipro_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE Alumipro_db;
```

```
CREATE TABLE IF NOT EXISTS usuarios (...);
CREATE TABLE IF NOT EXISTS clientes (...);
CREATE TABLE IF NOT EXISTS productos (...);
CREATE TABLE IF NOT EXISTS ventas (...);
CREATE TABLE IF NOT EXISTS detalle_venta (...);
```

Comandos principales de instalación y ejecución

```
# Backend
cd apps/backend
mvn clean test
mvn spring-boot:run
```

```
# Frontend
cd apps/frontend
npm install
npm run dev
```

```
# Base de datos en MySQL/XAMPP
mysql -u root < db/schema.sql
mysql -u root < db/seed.sql
```

```
# Base de datos con Docker Compose
docker compose up -d db
```

13. Configuración del backend

Propiedad	Valor usado	Descripción
server.port	8080 por defecto	Puerto donde se publica la API.
spring.datasource.url	jdbc:mariadb://localhost:3306/Alumipro_db	Cadena de conexión a MariaDB/MySQL.
spring.datasource.username	root	Usuario de base de datos en entorno local.
spring.datasource.password	vacío en XAMPP local	Debe configurarse en ambientes reales.
spring.jpa.hibernate.ddl-auto	update	Sincronización de entidades con base de datos durante desarrollo.
app.jwt.secret	Variable JWT_SECRET	Se debe cambiar en producción.
app.jwt.expirationMinutes	480	Tiempo de expiración del token.

14. Servicios API REST

Método	Ruta	Seguridad	Función
POST	/api/auth/login	Público	Autenticación y generación de JWT.
GET	/api/auth/me	Autenticado	Consulta del usuario autenticado.
GET/POST/PUT/DELETE	/api/clientes	ADMIN / VENDEDOR según operación	CRUD de clientes.
GET	/api/productos	Autenticado	Consulta de productos.
POST/PUT/DELETE	/api/productos	ADMIN	Gestión administrativa de productos.
POST	/api/ventas	ADMIN / VENDEDOR	Registro de venta y descuento de stock.
GET	/api/ventas	Autenticado	Consulta de histórico.
GET	/api/ventas/{id}	Autenticado	Detalle de venta.
GET/POST/PUT/DELETE	/api/usuarios	ADMIN	Administración de usuarios y roles.
GET	/api/movil/notificaciones	Autenticado	Notificaciones operativas para app móvil.

15. Seguridad y control de roles

La seguridad del backend se implementa con Spring Security en modo stateless. El endpoint `/api/auth/login` autentica credenciales y genera un token JWT. En cada petición protegida, el filtro `JwtAuthFilter` valida el encabezado `Authorization: Bearer <token>` y carga el usuario autenticado en el contexto de seguridad.

Rol	Permisos
ADMIN	Puede gestionar usuarios, productos, clientes, ventas e histórico. Tiene acceso a rutas administrativas.
VENDEDOR	Puede registrar ventas, consultar información operativa, gestionar clientes según reglas permitidas y consultar productos, pero no administra usuarios ni crea productos.

16. Componentes backend

Componente	Responsabilidad
ApiAuthController	Autenticación, generación de JWT y consulta de usuario autenticado.
ApiClienteController	Operaciones REST de clientes.
ApiProductoController	Operaciones REST de productos.
ApiVentaController	Registro, histórico y detalle de ventas.
ApiUsuarioController	Administración de usuarios.
ApiMovilController	Consulta de notificaciones para la aplicación móvil.
ClienteService	Validaciones y reglas de negocio de clientes.
ProductoService	Validaciones, consulta y gestión de inventario.
VentaService	Registro transaccional de ventas, cálculo de totales y descuento de stock.
Repositories JPA	Acceso a datos mediante interfaces <code>JpaRepository</code> .

17. Componentes frontend

Componente	Responsabilidad
router.tsx	Define rutas públicas, rutas protegidas y rutas exclusivas de administrador.
RequireAuth	Protege rutas que requieren sesión activa.
RequireAdmin	Restringe módulos administrativos, como productos administrativos y usuarios.
Sidebar	Menú lateral; oculta Usuarios cuando el usuario no es ADMIN.
DashboardPage	Métricas de productos, clientes, ventas e ingresos.
ClientesListPage / ClienteFormPage	Listado, búsqueda y formulario de clientes.
ProductosListPage / ProductoFormPage	Listado, búsqueda y formulario de productos.
VentaCreatePage	Flujo de selección de cliente, producto, cantidad, ítems y registro de venta.
VentasHistoryPage	Histórico y detalle de ventas.
UsuariosPage	Administración de usuarios desde rol ADMIN.

18. Procedimiento técnico de instalación local

1. Clonar o descargar el repositorio oficial Alumipro_INC.
2. Iniciar MySQL desde XAMPP o levantar el contenedor definido en docker-compose.yml.
3. Importar db/schema.sql y db/seed.sql sobre la base Alumipro_db.
4. Abrir apps/backend en IntelliJ IDEA como proyecto Maven.
5. Validar application.properties: puerto 8080, datasource y variables JWT.
6. Ejecutar mvn clean test para verificar pruebas backend.
7. Ejecutar mvn spring-boot:run o la clase AlumiproAa3Application.
8. Abrir apps/frontend en terminal, ejecutar npm install y luego npm run dev.
9. Ingresar en el navegador a <http://localhost:5173> y validar login con credenciales de prueba.
10. Ejecutar pruebas funcionales: clientes, productos, ventas, histórico y usuarios.

19. Pruebas técnicas recomendadas

Área	Comando / herramienta	Validación
Backend	mvn test	Ejecuta pruebas unitarias y de controladores.
Backend	mvn package	Compila y empaqueta el proyecto.
Frontend	npm run lint	Valida reglas de calidad estática.
Frontend	npm run test	Ejecuta pruebas automatizadas frontend.
Frontend	npm run build	Genera paquete de producción.
API	Colección Postman	Valida endpoints REST y reglas de autenticación.
Funcional	Pruebas manuales	Login, roles, CRUD, ventas, stock e histórico.

20. Consideraciones de soporte y mantenimiento

- Cambiar JWT_SECRET en ambientes diferentes a desarrollo.
- No usar contraseñas vacías de base de datos en producción.
- Mantener respaldo periódico de Alumipro_db.
- Versionar cambios de scripts SQL y aplicar migraciones de forma controlada.
- Validar reglas de stock antes de modificar VentaService.
- Verificar compatibilidad de Spring Boot, Java, Node y versiones de dependencias antes de actualizar.
- Mantener documentación de endpoints y colección Postman actualizadas.

21. Relación con la documentación de usuario

El manual de usuario de Alumipro se maneja como documento independiente en la evidencia.

Este manual técnico se complementa con esa documentación, ya que describe internamente

cómo está construido, instalado y configurado el software, mientras el manual de usuario explica el uso operativo de las funciones del sistema.

22. Conclusiones

El sistema Alumipro cuenta con una arquitectura web organizada por capas, separando claramente presentación, servicios REST, lógica de negocio, seguridad y persistencia. Esta organización facilita el mantenimiento, las pruebas y la implantación del sistema.

La base de datos relacional permite representar adecuadamente clientes, productos, usuarios, ventas y detalle de ventas, manteniendo integridad mediante llaves foráneas, restricciones e índices.

El uso de JWT y control de roles permite restringir funcionalidades críticas, diferenciando correctamente las capacidades del administrador y del vendedor dentro del sistema.

El manual técnico deja documentados los elementos principales requeridos para la evidencia: prerequisites, frameworks, estándares, casos de uso, modelo entidad relación, diccionario de datos, scripts de instalación y diagrama de componentes.

23. Bibliografía

- Servicio Nacional de Aprendizaje - SENA. (s. f.). Guía de aprendizaje 10: Implementar la solución de software de acuerdo con los requisitos de operación y modelos de referencia.
- Departamento Nacional de Planeación. (s. f.). Guía para la elaboración de manual técnico y de operación de los sistemas de información.

- Proyecto ALUMIPRO. (2026). Repositorio técnico Alumipro_INC: README.md, db/schema.sql, db/seed.sql, application.properties, controladores API y documentación docs/01-endpoints.md.
- Oracle. (s. f.). Documentación de MySQL.
- Spring. (s. f.). Documentación de Spring Boot, Spring Security y Spring Data JPA.
- React. (s. f.). Documentación oficial de React.